

AVP Seminar 2주차

기본 명령어, Service, Topic, Action

Mingyu Choi

Architecture and Compiler for Embedded System LAB.

School of Electronics Engineering, KNU, KOREA

2026-06-23



목차

- ROS2 기본 터미널 명령어
- Service
- Topic
- Action

ROS2 기본 터미널 명령어

- Jupyter Notebook, markdown 등 강의 속 여러 명령어들이 있었으나 실질적으로 사용되지 않을 것 같아서 제외

- bashrc 설정

bashrc란 터미널을 새로 열 때마다 자동으로 실행되는 설정 파일으로, 홈 디렉토리에 숨김파일로 존재한다.

새 터미널 창을 열면 bash가 이 파일을 위에서부터 아래로 한 줄씩 읽어서 실행한다.

ROS2를 쓰려면 매번 터미널에서 `source /opt/ros/humble/setup.bash` 이 명령어를 실행해야하는데,

.bashrc 코드 맨 아래에 해당 `source /opt/ros/humble/setup.bash` 명령어를 적어두면 터미널을 열 때 자동으로 ROS2가 준비된 상태가 된다.

ROS2 기본 터미널 명령어

- home 파일에 있는 bashrc 열기 (vscode 활용)

code ~/.bashrc

- .bashrc 만

echo "ROS2

source /opt

(echo 명령

-> 터미널을

```
admin2@admin: ~ 111x28
ROS2 humble is activated!
admin2@admin:~$
```

ROS2 기본 터미널 명령어

- bashrc에서 alias 설정

이런 것들이 터미널에 자주 나오게 되어 드루킹이 되는 것

```
admin2@admin: ~ 111x28
admin2@admin:~$ humble
ROS2 humble is activated
admin2@admin:~$
```

-> humble 명령이 입력 시 ROS2가 자동 실행됨

ROS2 기본 명령어

- Node의 실행

Node

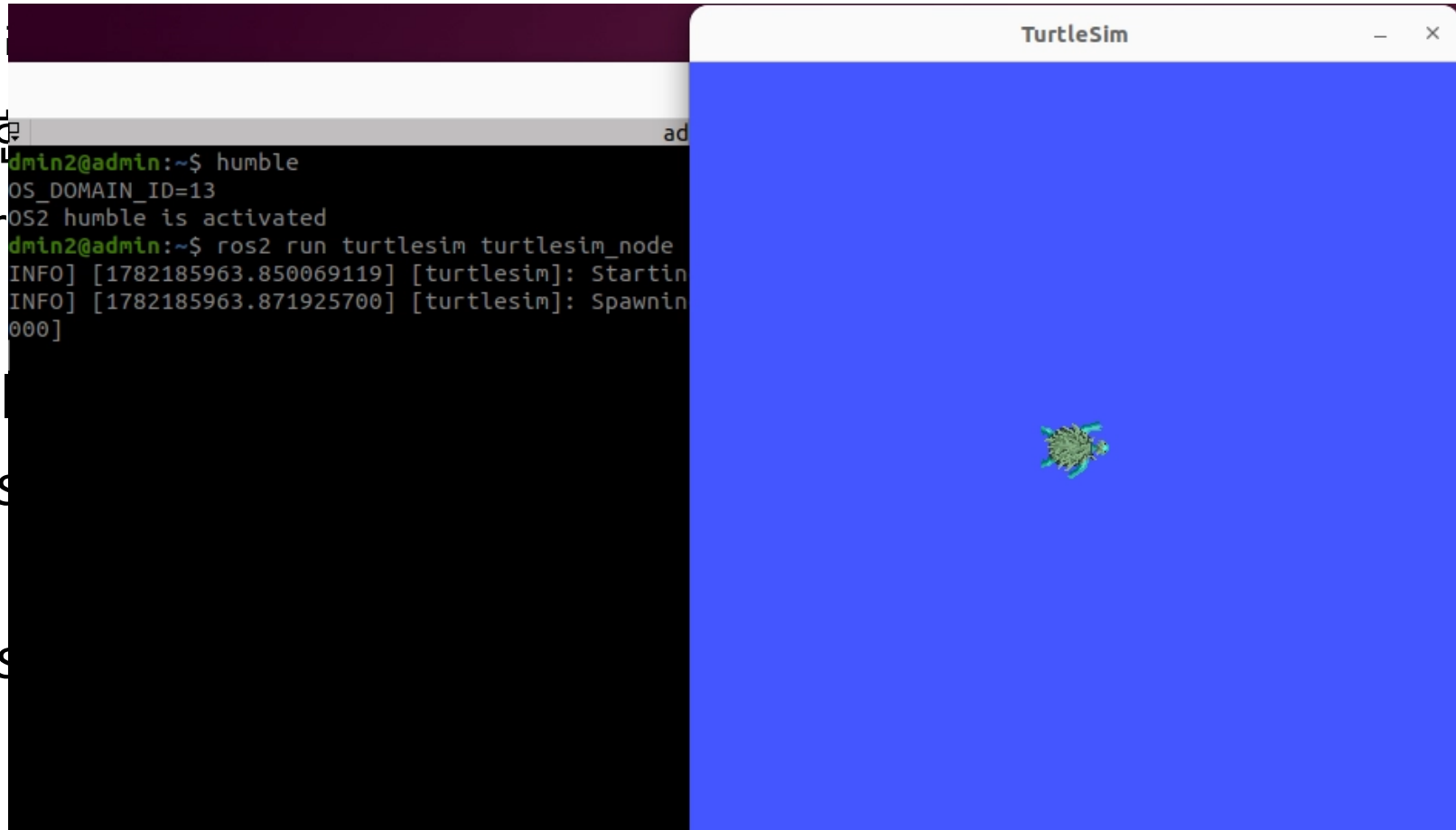
(특정함

ros2 r

- Turtle

turtles

turtles



ROS2 기본 명령어

- 현재 실행 중인 node list 확인

ros2 node list

-> 방금 실행한 /turtlesim node 존

```
admin2@admin:~$ humble
ROS_DOMAIN_ID=13
ROS2 humble is activated
admin2@admin:~$ ros2 node list
/turtlesim
admin2@admin:~$
```

- 해당 node의 정보 확인

ros2 node info /turtlesim

```
admin2@admin:~$ ros2 node info /turtlesim
/turtlesim
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/color_sensor: turtlesim/msg/Color
  /turtle1/pose: turtlesim/msg/Pose
Service Servers:
  /clear: std_srvs/srv/Empty
  /kill: turtlesim/srv/Kill
  /reset: std_srvs/srv/Empty
  /spawn: turtlesim/srv/Spawn
  /turtle1/set_pen: turtlesim/srv/SetPen
  /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
  /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
  /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /turtlesim/get_parameters: rcl_interfaces/srv/GetParameters
  /turtlesim/list_parameters: rcl_interfaces/srv/ListParameters
  /turtlesim/set_parameters: rcl_interfaces/srv/SetParameters
  /turtlesim/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Service Clients:

Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:
```

ROS2 기본 명령어 - Service

- Service



Client 노드가 Server node에게 무언가 요청(request)하고,
Server 노드는 해당 요청을 수행하여 결과를 응답(response)한다

ROS2 기본 명령어 - Service

- 서비스 목록 확인

ros2 service list

-> turtlesim node가 제공하는 service list이다.

```
admin2@admin:~$ humble
ROS_DOMAIN_ID=13
ROS2 humble is activated
admin2@admin:~$ ros2 service list
/clear
/kill
/reset
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/describe_parameters
/turtlesim/get_parameter_types
/turtlesim/get_parameters
/turtlesim/list_parameters
/turtlesim/set_parameters
/turtlesim/set_parameters_atomically
admin2@admin:~$
```

- 해당 Service의 datatype 확인

ros2 service type /turtle1/teleport_absolute

```
admin2@admin:~$ ros2 service type /turtle1/teleport_absolute
turtlesim/srv/TeleportAbsolute
admin2@admin:~$
```

request할 때와 response할 때 datatype을 선언하므로 해당 Service의 datatype을 확인해야한다.

-> turtle1에 있는 teleport_absolute의 data type :

turtlesim/srv/TeleportAbsolute

ROS2 기본 명령어 - Service

- 서비스 data type 이외 definition을 확인하고 싶다면 ?
-> 해당 정보들을 기록해놓은 srv 파일을 열어본다.

ros2 interface show turtlesim/srv/TeleportAbsolute

```
admin2@admin:~$ ros2 interface show turtlesim/srv/TeleportAbsolute
float32 x
float32 y
float32 theta
---
```

- > - - - 는 request와 response를 분리해주어 사진과 같이
float32 x, float32 y, float32 theta는 request할 때 필요한 datatype
이고 , 밑에는 response할 때 필요한 datatype이다

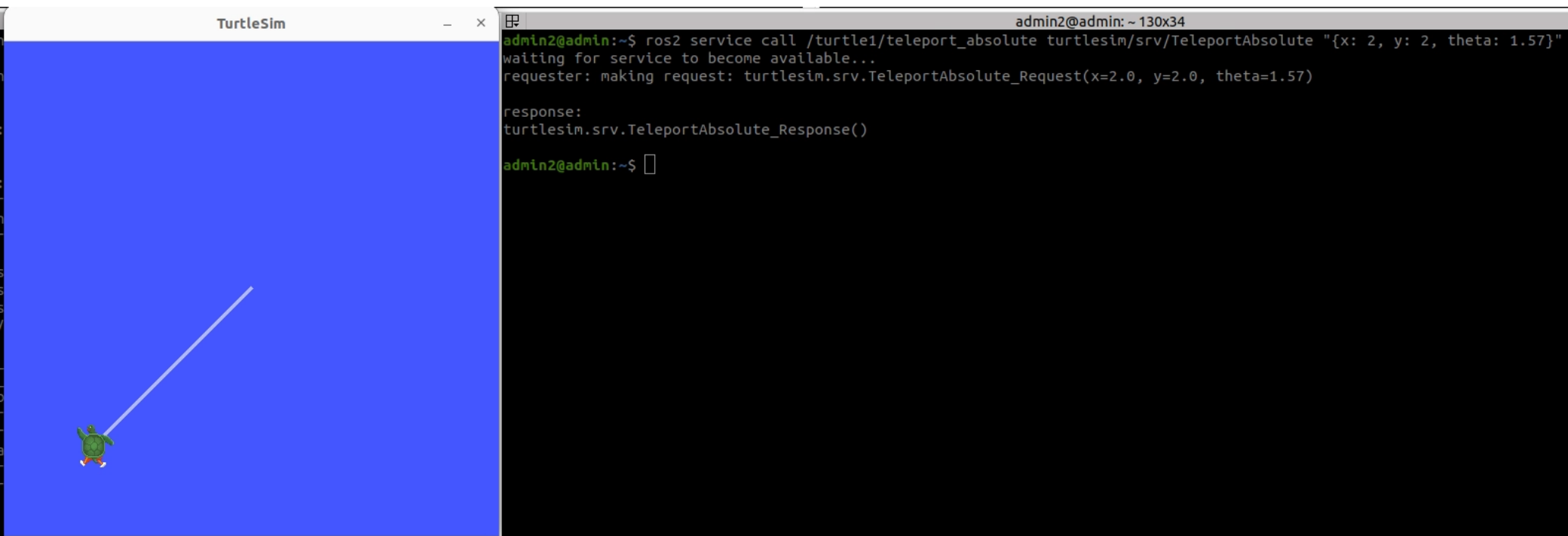
→ TeloportAbsolute datatype은 request할 때는 실수형 3개
x,y,theta 값을 주면서 request하고, response 영역은 없다. (비워둠)

ROS2 기본 명령어 - Service

- Service call 명령어

ros2 service call /turtle1/teleport_absolute
turtlesim/srv/TeleportAbsolute "{x: 2, y: 2, theta: 1.57}"

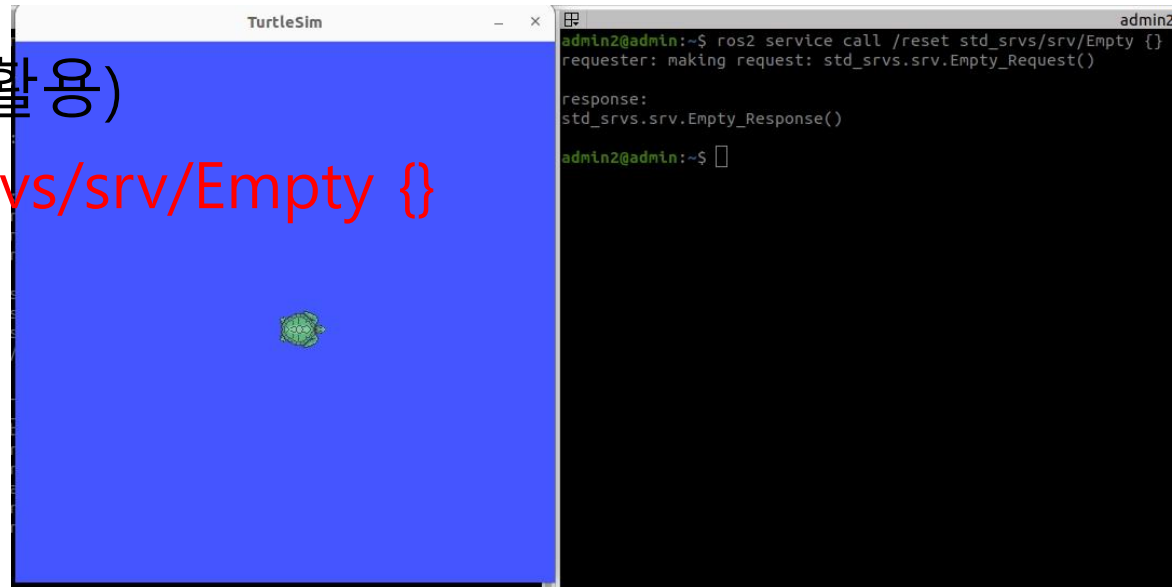
-> ros2 service call + "서비스 이름" + "서비스 datatype" + "필요로 하는 값"



ROS2 기본 명령어 - Service

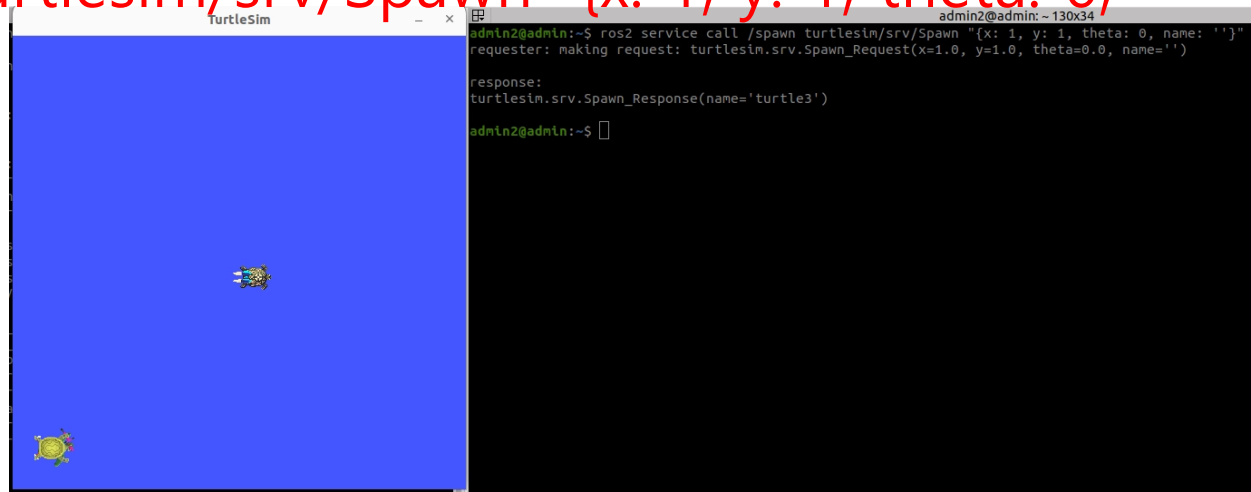
- 위치 원위치 (reset service 활용)

`ros2 service call /reset std_srvs/srv/Empty {}`



- 거북이 소환 (spawn service 활용)

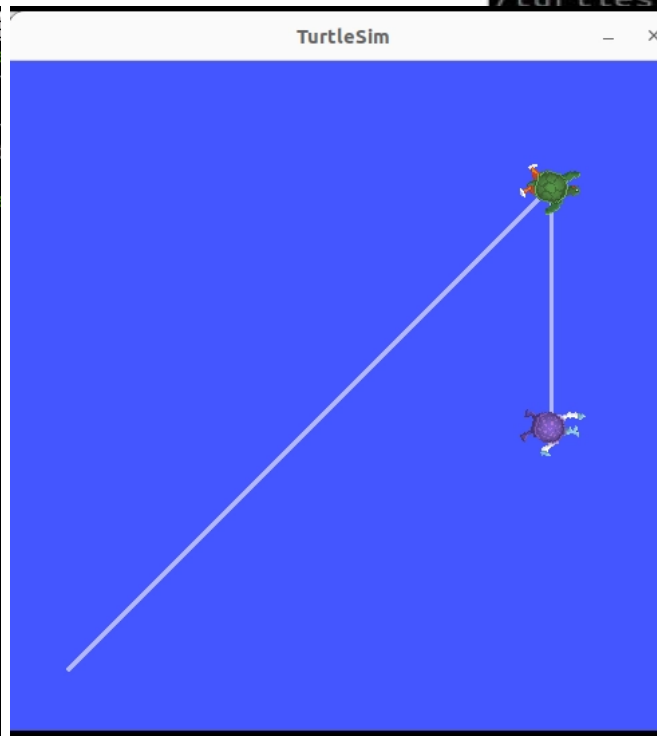
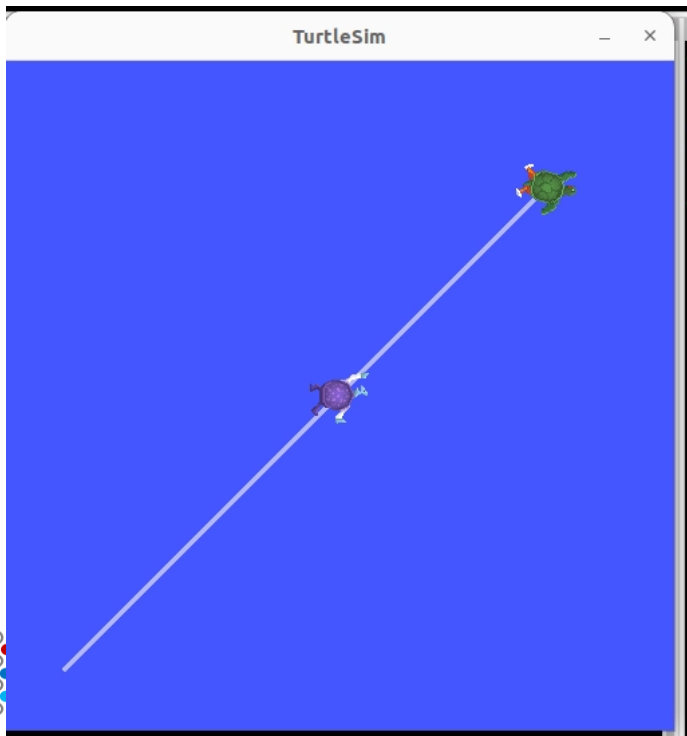
`ros2 service call /spawn turtlesim/srv/Spawn "{x: 1, y: 1, theta: 0, name: ''}"`



ROS2 기본 명령어 - Service

- 다시 service list 확인
-> turtle2에 대한 service가 추가 생성
(각 거북이 따로 제어 가능)

```
admin2@admin:~$ ros2 service list
/clear
/kill
/reset
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtle2/set_pen
/turtle2/teleport_absolute
/turtle2/teleport_relative
/turtle3/set_pen
/turtle3/teleport_absolute
/turtle3/teleport_relative
/turtlesim/describe_parameters
/turtlesim/get_parameter_types
/turtlesim/get_parameters
/turtlesim/list_parameters
/turtlesim/set_parameters
/turtlesim/set_parameters_atomically
```



ROS2 기본 명령어 - Topic

- 어떤 Topic을 목적 대상 없이 Publish하는 Publisher 노드가 존재하고, 해당 Topic을 구독하는 node들이 subscribe한다.



Topic은 노드들이 메시지를 계속 주고받는, 이름이 붙은 통로이다.
그리고 토픽은 여러 노드가 같은 토픽에 동시에 붙을 수 있다.

ROS2 기본 명령어 - Topic

- Topic 목록 확인

`ros2 topic list`

```
admin2@admin:~$ ros2 topic list
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

- Topic type 확인 (Topic이 사용하는 메시지의 data type)

`ros2 topic type /turtle1/pose` (/turtle1/pose topic의 data type 확인)

```
admin2@admin:~$ ros2 topic type /turtle1/pose
turtlesim/msg/Pose
admin2@admin:~$
```

-> turtlesim/msg/Pose

- Publish와 Subscribe

`ros2 topic list -v`

```
admin2@admin:~$ ros2 topic list -v
Published topics:
* /parameter_events [rcl_interfaces/msg/ParameterEvent] 2 publishers
* /rosout [rcl_interfaces/msg/Log] 2 publishers
* /turtle1/color_sensor [turtlesim/msg/Color] 1 publisher
* /turtle1/pose [turtlesim/msg/Pose] 1 publisher

Subscribed topics:
* /parameter_events [rcl_interfaces/msg/ParameterEvent] 2 subscribers
* /turtle1/cmd_vel [geometry_msgs/msg/Twist] 1 subscriber
```

ROS2 기본 명령어 - Topic

- ros2 topic info

ros2 topic info /turtle1/pose

datatype : turtlesim/msg/Pose

Publisher count : 1

Subscription count : 0

```
admin2@admin:~$ ros2 topic info /turtle1/pose
Type: turtlesim/msg/Pose
Publisher count: 1
Subscription count: 0
```

- ros2 interface show

```
admin2@admin:~$ ros2 interface show turtlesim/msg/Pose
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
```

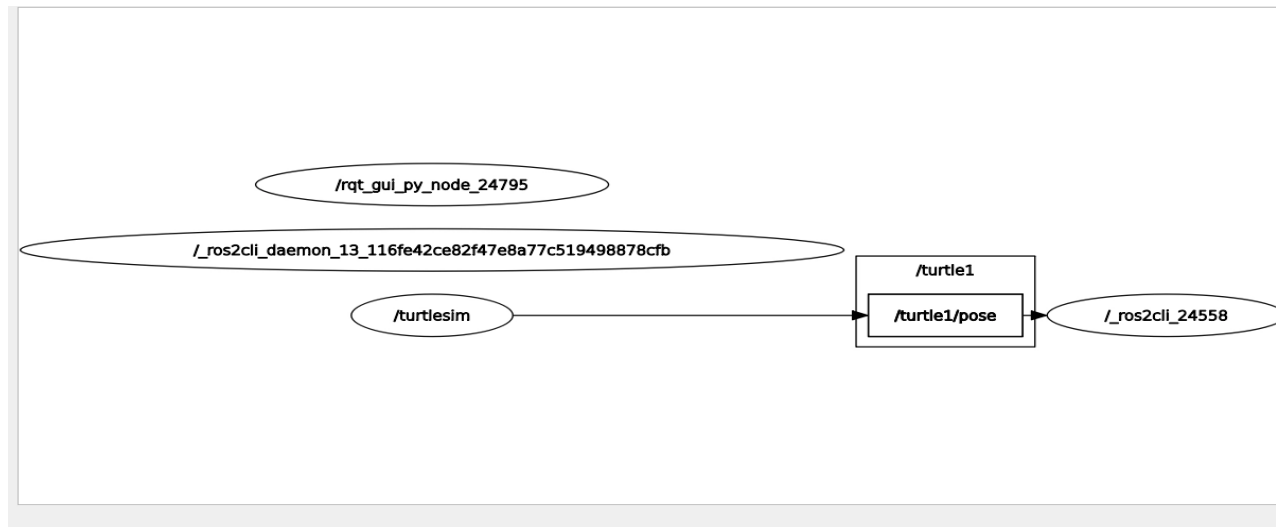
-> x,y,theta 정보를 받고 속도 성분까지 가지고 있다.

ROS2 기본 명령어 - Topic

- Topic 구독(Subscribe)하는 법

`ros2 topic echo /turtle1/pose`

`ros2 topic echo /turtle1/pose --once` (한 번만 확인)



-> `rqt_graph`를 보면 `ros2cli_24558`은 터미널로,
`/turtlesim` 노드에서 터미널로 `/turtle1/pose` 토픽이 전달된다.

ROS2 기하 매커니즘

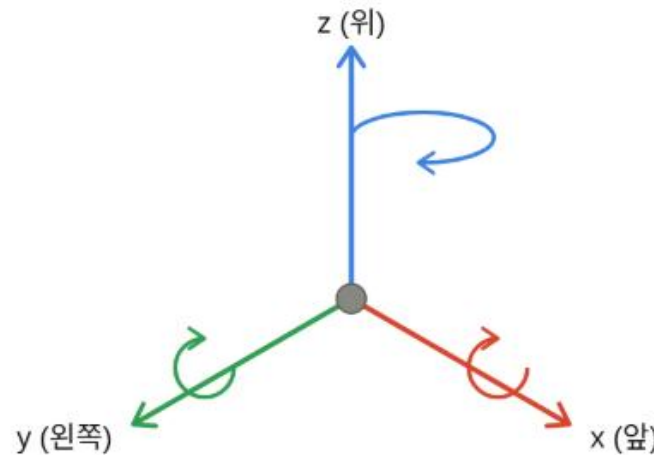
- 거북이 즉

ros2 topic

-> /turtle1

- cmd_vel

ros2 interface show geometry_msgs/msg/Twist



geometry_msgs/Twist

linear (이동)

■ linear.x · 앞뒤

■ linear.y · 좌우

■ linear.z · 상하

angular (회전)

■ angular.x · roll

■ angular.y · pitch

■ angular.z · yaw

turtlesim은 이 둘만 사용:

linear.x · angular.z

parameterEvent]

st]

lor]

Twist]

```
admin2@admin:~$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3  linear
  float64 x
  float64 y
  float64 z
Vector3  angular
  float64 x
  float64 y
  float64 z
```

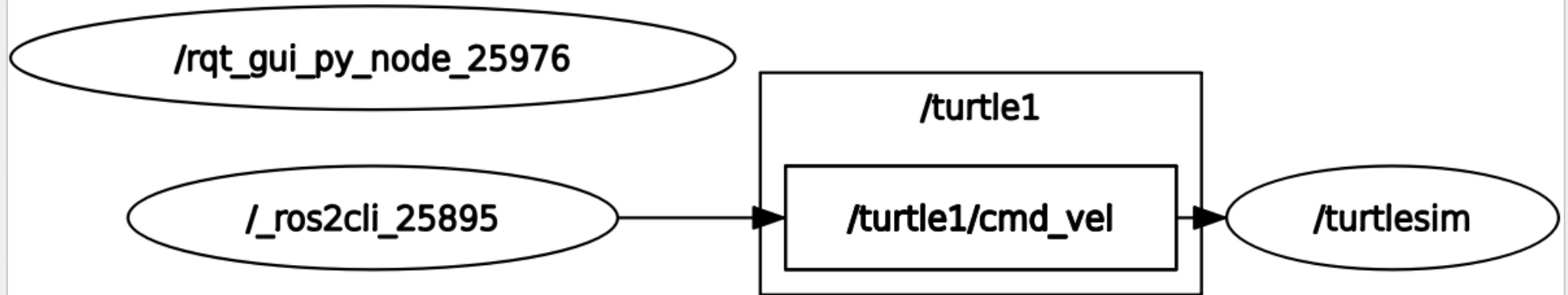
-> 3차원 벡터 linear(x,y,z 축 각각 방향의 속도 값) 벡터와



Angular(x,y,z 축을 중심으로 한 회전 값) 벡터로 구성

ROS2 기본 명령어 - Topic

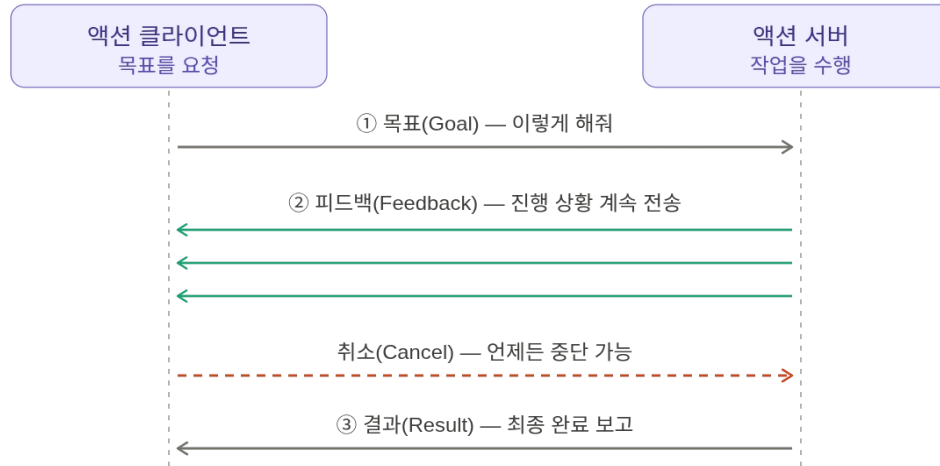
- Topic 발행하는 법 (ros2 topic pub <-> 구독 : ros2 topic echo)



> rqt_graph에서 구독할 때와 반대로 터미널 노드에서 cmd_vel
Topic을 발행하여 /turtlesim이 구독하는 상태를 확인할 수 있다

ROS2 기본 명령어 - Action

- Action



action을 수행하는 Server가 있고 이 action을 client가 목표를 요청하면, 그 목표를 이수하기 까지 중간과정을 feedback으로 전송하며 최종 결과를 보고한다.

목표-결과-취소 (Service로 구성)

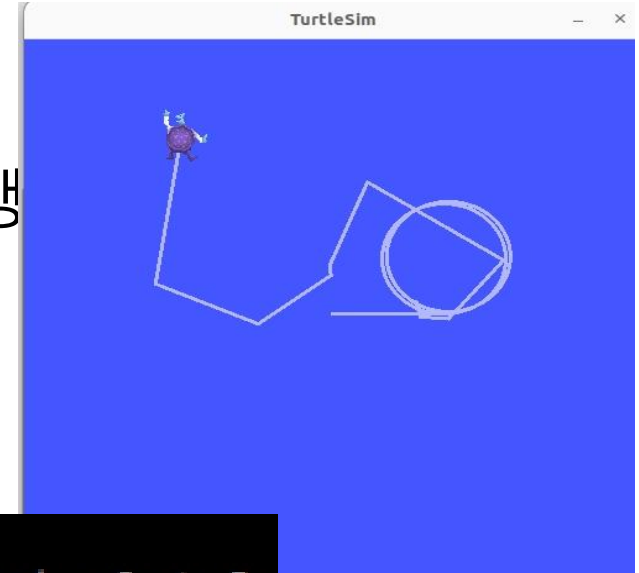
목표를 보냈으면 받아줬는지(수락/거부)를 반드시 확인해야 한다. 따라서 확실한 1:1 응답이 필요하다

피드백-상태 (Topic으로 구성)

진행 상황은 작업 내내 수십번씩 보내기에 매번 받아줬는지 확인 받을 필요 없이 계속 흘려보내기만 하면 된다. 따라서 응답없이 계속 발송하는 토픽으로 구성

ROS2 기본 명령어 - Action

- turtlesim_node와 turtle_teleop_key 노드 실행
-> 화살표 키로 막 움직일 수 있게 됨



- action list 확인

```
admin2@admin:~$ ros2 action list -t
/turtle1/rotate_absolute [turtlesim/action/RotateAbsolute]
```

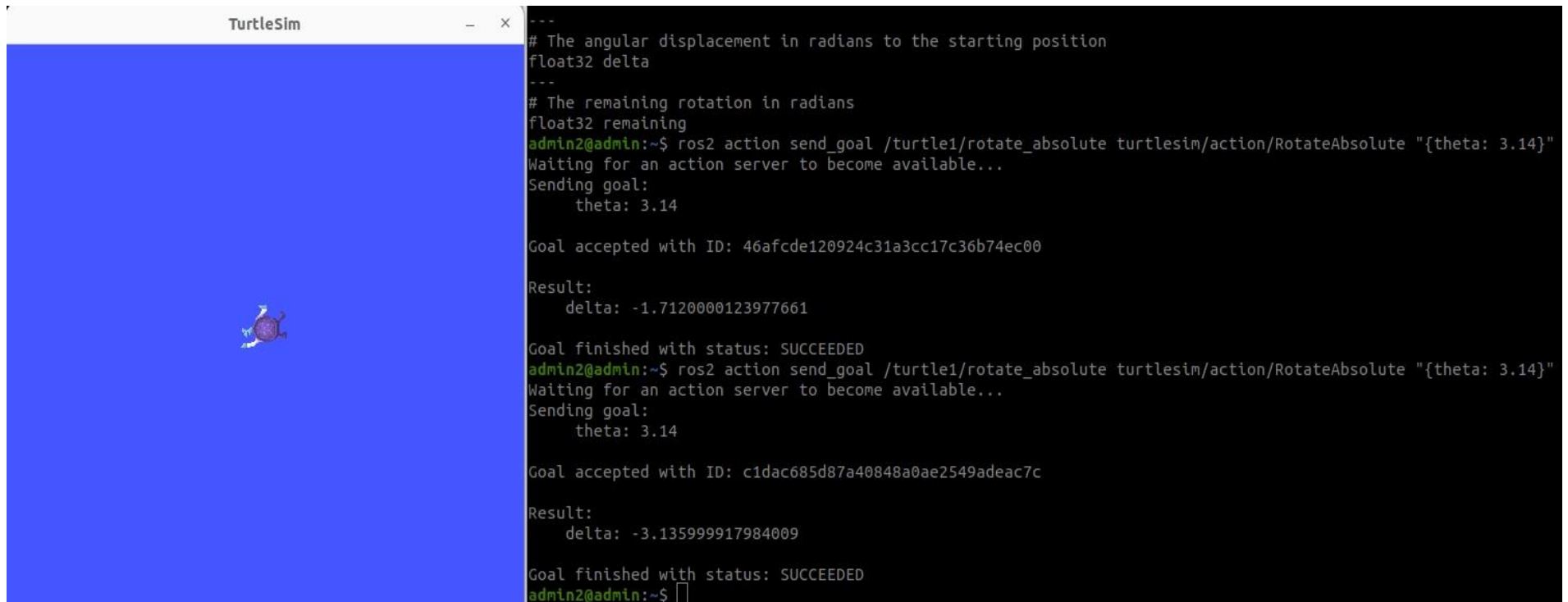
```
admin2@admin:~$ ros2 interface show turtlesim/action/RotateAbsolute
# The desired heading in radians
float32 theta
---
# The angular displacement in radians to the starting position
float32 delta
---
# The remaining rotation in radians
float32 remaining
```

> action은 1. 어디로 갈건지(목표) 2. 어떻게 됐는지(결과) 3.현재 얼마나 남았는지를 구분한다.

ROS2 기본 명령어 - Action

- 액션의 목표 지정 (send_goal)

ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 3.14}"

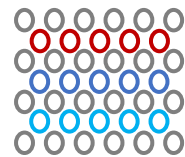


The image shows a screenshot of a ROS2 environment. On the left is a window titled "TurtleSim" with a blue background and a small turtle icon. On the right is a terminal window with a black background and white text. The terminal shows the execution of the command `ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 3.14}"` twice. The first execution shows the goal being accepted with ID `46afcde120924c31a3cc17c36b74ec00` and the result `delta: -1.7120000123977661`. The second execution shows the goal being accepted with ID `c1dac685d87a40848a0ae2549adeac7c` and the result `delta: -3.135999917984009`. Both goals finished with status `SUCCEEDED`.

```
---  
# The angular displacement in radians to the starting position  
float32 delta  
---  
# The remaining rotation in radians  
float32 remaining  
admin2@admin:~$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 3.14}"  
Waiting for an action server to become available...  
Sending goal:  
  theta: 3.14  
  
Goal accepted with ID: 46afcde120924c31a3cc17c36b74ec00  
  
Result:  
  delta: -1.7120000123977661  
  
Goal finished with status: SUCCEEDED  
admin2@admin:~$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 3.14}"  
Waiting for an action server to become available...  
Sending goal:  
  theta: 3.14  
  
Goal accepted with ID: c1dac685d87a40848a0ae2549adeac7c  
  
Result:  
  delta: -3.135999917984009  
  
Goal finished with status: SUCCEEDED  
admin2@admin:~$
```

Q & A

Thank you for your attention



Architecture and Compiler
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU
ACE Lab. (cmg031107@naver.com)